

# lab2 实验报告

---

## Thinking

### 2.1

**Q:**

请根据上述说明，回答问题：在编写的 C 程序中，指针变量中存储的地址被视为虚拟地址，还是物理地址？MIPS 汇编程序中 lw 和 sw 指令使用的地址被视为虚拟地址，还是物理地址？

**A:**

虚拟地址

### 2.2

**Q:**

- 从可重用性的角度，阐述用宏来实现链表的好处。
- 查看实验环境中的 `/usr/include/sys/queue.h`，了解其中单向链表与循环链表的实现，比较它们与本实验中使用的双向链表，分析三者插入与删除操作上的性能差异

**A:**

- 使用宏来实现链表能够实现代码的复用，且一定程度上提高了可读性，同时便于维护
- - 单向链表删除某个节点或需要在某一项前面插入节点，需要从头遍历到该项，平均复杂度  $O(n/2)$ ，但是如果需要在某一项后面插入节点，则直接插入，时间复杂度  $O(1)$ 。
  - 循环链表与单向链表基本相同，不同的是该链表能够直接在链表尾部插入节点。
  - 双向链表：能够直接插入，删除链表的某一项，时间复杂度  $O(1)$ ，但如果想要访问链表尾节点，仍需遍历整个链表，平均时间复杂度  $O(n/2)$ 。

### 2.3

**Q:**

请阅读 `include/queue.h` 以及 `include/pmap.h`，将 `Page_list` 的结构梳理清楚，选择正确的展开结构。

**A:**

C

### 2.4

**Q:**

- 请阅读上面有关 TLB 的描述，从虚拟内存和多进程操作系统的实现角度，阐述 ASID 的必要性。
- 请阅读 MIPS 4Kc 文档《MIPS32® 4K™ Processor Core Family Software User's Manual》的 Section 3.3.1 与 Section 3.4，结合 ASID 段的位数，说明 4Kc 中可容纳不同的地址空间的最大数量。

**A:**

- 不同的进程有各自的页表和虚拟空间，同一个虚拟地址可能在不同的进程下映射到不同的物理地址，ASID可以区分不同的进程，从而确保映射到正确的物理地址
- ASID占据了8位，故最多支持 $2^8$ 即256个不同的虚拟地址

## 2.5

### Q:

- tlb\_invalidate 和 tlb\_out 的调用关系？
- 请用一句话概括tlb\_invalidate 的作用。
- 逐行解释tlb\_out 中的汇编代码。

### A:

- tlb\_invalidate函数中调用了tlb\_out函数。
- 讲对应进程中虚拟地址对应的TLB表项无效化

- ```
LEAF(tlb_out) #叶函数宏
.set noreorder
mfc0    t0, CP0_ENTRYHI #取出现在寄存器中的EntryHi
mtc0    a0, CP0_ENTRYHI #将传入参数写入EntryHi
nop
/* Step 1: Use 'tlbp' to probe TLB entry */
/* Exercise 2.8: Your code here. (1/2) */
tlbp    #根据写入的，即参数a0，查找TLB中对应的表项，并把索引存储在Index寄存器中
nop
/* Step 2: Fetch the probe result from CP0.Index */
mfc0    t1, CP0_INDEX # 将Index寄存器的值传入t1寄存器
.set reorder
bltz    t1, NO_SUCH_ENTRY #如果这个值小于0，那么说明TLB中没有找到a0对应的项，跳转到对应的位置
.set noreorder
mtc0    zero, CP0_ENTRYHI #否则，说明TLB中有对应的项，则把EntryHi和EntryLo置为零
mtc0    zero, CP0_ENTRYLO0
mtc0    zero, CP0_ENTRYLO1
nop
/* Step 3: Use 'tlbwi' to write CP0.EntryHi/Lo into TLB at CP0.Index */
/* Exercise 2.8: Your code here. (2/2) */
tlbwi   #然后写入索引指定的TLB表项中，即完成该表项的重置
nop
.set reorder
NO_SUCH_ENTRY:
mtc0    t0, CP0_ENTRYHI #恢复现场
j       ra #返回
END(tlb_out)
```

## 2.6

### Q:

请结合 Lab2 开始的 CPU 访存流程与下图中的 Lab2 用户函数部分，尝试将函数调用与CPU访存流程对应起来，思考函数调用与CPU访存流程的关系。

**A:**

CPU 访存 TLB 缺失 → 硬件触发异常 → `do_tlb_refill` → `_do_tlb_refill` → `page_lookup/passive_alloc` 查页表并分配物理页 → 将结果写入 TLB → 异常返回后重新执行访存指令。

2.7

**Q:**

简单了解并叙述X86体系结构中的内存管理机制，比较X86和MIPS 在内存管理上的区别。

**A:**

### x86体系结构中的内存管理机制

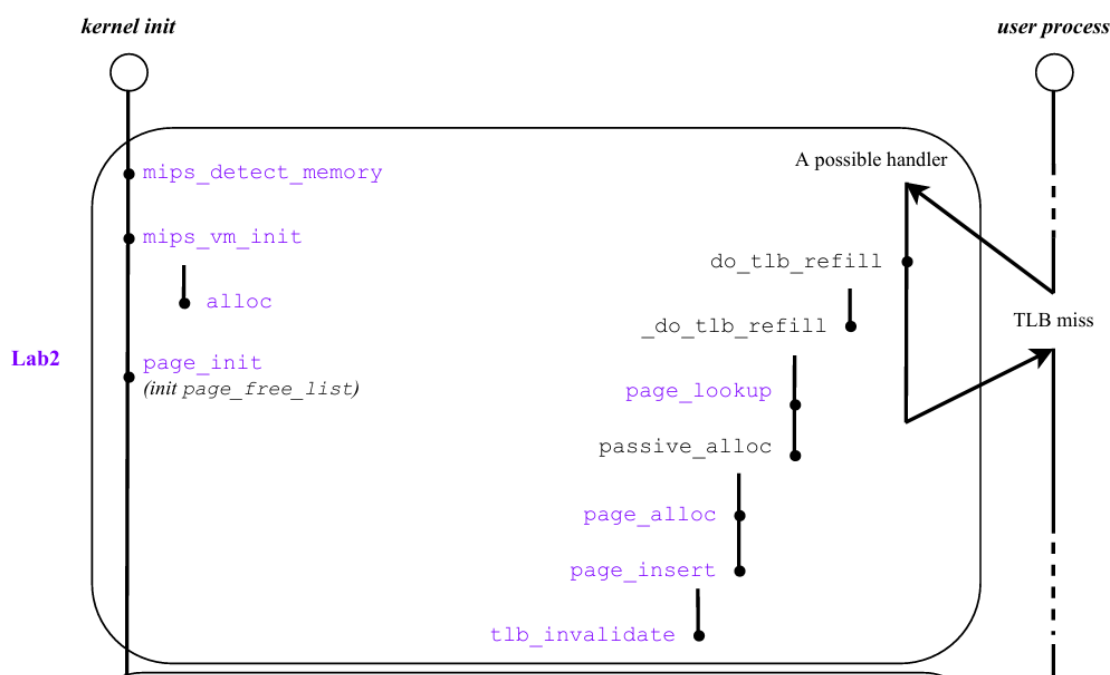
x86 采用分段 + 分页两级机制，逻辑地址经分段转为线性地址，再由分页转为物理地址；现代 x86 多使用平坦模式（分段基址为 0），实际依赖多级页表（如 4KB 页）完成地址映射与权限控制。

### x86与MIPS在内存管理上的区别

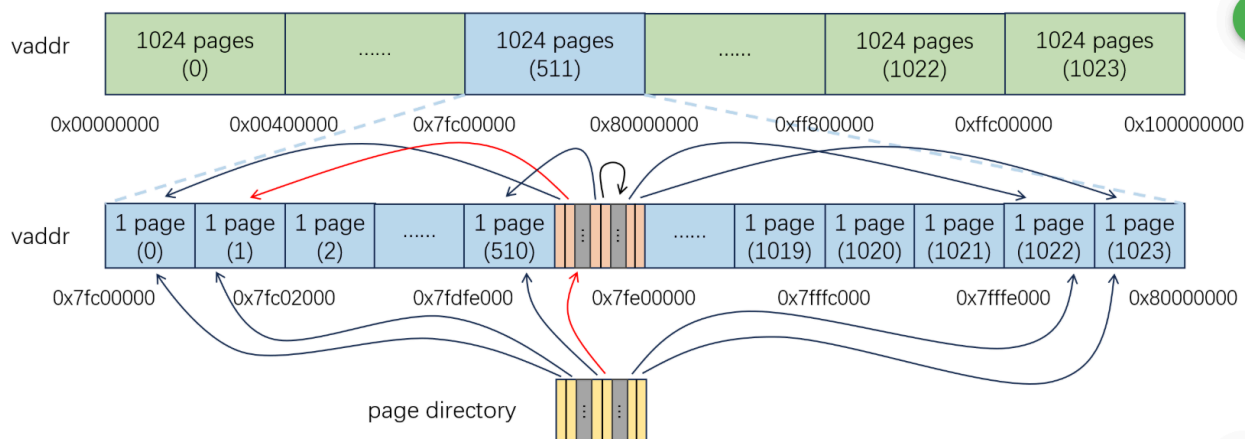
1. x86 支持分段与分页，MIPS 仅支持分页。
2. x86 硬件自动遍历页表填充 TLB；MIPS TLB 缺失由软件（操作系统）处理。
3. x86 硬件机制复杂，功能更丰富；MIPS 硬件简洁，内存管理依赖软件实现。

## 难点分析

1. 大量的宏，复杂的结构体设计增加了阅读难度
2. `page`、`va`、`pa`三者的来回转换，让人摸不着头脑
3. 函数之间的调用，页表建立的流程复杂，需要花大量时间理解



#### 4. 理论课讲上的自映射机制



## 实验体会

本次实验的难度较大，完全理解和掌握需要花不少功夫。实验总体分为页表机制建立与TLB重填两大部分。tlb重填的工作量较小，实现起来不是很困难，但C语言与MIPS的交互，同时涉及到了CP0的异常处理，是这一块最大的挑战。页表机制建立才是最困难的，其中涉及到了大量的宏，复杂的结构体设计以及page、va、pa三者的来回转换，很容易让人摸不着头脑，直到上机时本人还未完全掌握，故extra的状况惨烈，这块真的需要多花时间!!!